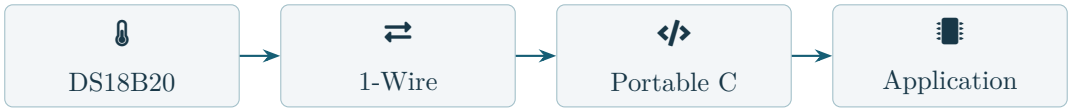
درایورهای سیستم‌های نهفته

یاد بگیریم، بسازیم، دوباره استفاده کنیم

دفتر آموزش عملی و گزارش پروژه‌ها

از اولین خواندن دما تا کتابخانه‌ای برای استفاده در پروژه‌های دیگر.



مسیر یادگیری

مدار را وصل کنید. دما را بخوانید. کار هر تابع را بفهمید. کتابخانه را به پروژهٔ خودتان ببرید.

پروژهٔ اول: DS18B20 / STM32F107VCT6 / PB10

Keil · PlatformIO · CubeMX

نویسنده: **علیرضا خواجه‌وندی**

نسخه: 2026-10-12

صفحات مجازی و راه‌های ارتباطی علیرضا خواجه‌وندی

وب‌سایت · گیت‌هاب · لینکدین

یوتیوب · تلگرام · اینستاگرام

فهرست مطالب

۱ مجموعه‌ای که پروژه‌به‌پروژه رشد می‌کند ۳

۱.۱ فهرست فعلی ۳

۲.۱ جای هر بخش در پروژه ۳

۳.۱ روش مطالعه و بازتولید ۴

۲ خواندن اولین دما با سنسور ۵

۱.۲ قرار است چه چیزی بسازیم؟ ۵

۲.۲ مرحلهٔ ۱: سه سیم و یک مقاومت را بررسی کنیم ۵

۳.۲ مرحلهٔ ۲: بازکردن، ساخت و دانلود در کیل ۶

۴.۲ مرحلهٔ ۳: Watch برنامهٔ در حال اجرا را نشان بدهد ۶

۵.۲ مرحلهٔ ۴: نمایش خراب است یا خواندن سنسور؟ ۸

۶.۲ مرحلهٔ ۵: دو تابع در پروژهٔ STM۳۲ خودمان ۸

۷.۲ مرحلهٔ ۶: نمونهٔ کامل اجرایی را بفهمیم ۱۰

۸.۲ مرحلهٔ ۷: تغییر پایه یا شروع از کپی پروژه ۱۱

۹.۲ مرحلهٔ ۸: توالی سنسور را بدون خواندن کل کد دنبال کنیم ۱۲

۱۰.۲ مرحلهٔ ۹: تابع قابل‌حمل را بخش‌به‌بخش بخوانیم ۱۳

۱۱.۲ مرحلهٔ ۱۰: وقتی خواستیم منتقل کنیم، آداپتر را بفهمیم ۱۴

۱۲.۲ مرحلهٔ ۱۱: برد واقعاً چه چیزی را ثابت کرد؟ ۱۵

۳ افزودن درایور مستقل بعدی ۱۶

۱.۳ شروع از کپی قالب پایه ۱۶

۲.۳ ترتیب توسعهٔ قابل تکرار ۱۶

۳.۳ افزودن فصل به همین گزارش ۱۷

دربارهٔ نویسنده و راه‌های ارتباطی ۱۹

۱ مجموعه‌ای که پروژه‌به‌پروژه رشد می‌کند

این مخزن را برای استفادهٔ دوباره از درایورهای سیستم‌های نهفته و آموزش روش ساخت آن‌ها نگه می‌دارم. هر قطعه یک پروژهٔ کامل دارد که می‌توان آن را به پوشه‌ای دیگر کپی کرد. خود کتابخانهٔ قابل‌حمل نیز به‌تنهایی قابل انتقال است؛ برای اتصال آن به میکروکنترلر دیگر یک آداپتر سخت‌افزاری کوچک نوشته می‌شود.

گزارش، مستند مشترک مجموعه است و وابستگی مشترک ساخت firmware نیست. یک نسخهٔ انگلیسی و یک نسخهٔ فارسی داریم. هر درایور جدید یک فصل دارد که مدار، کد، روش تست و وضعیت اعتبارسنجی همان پروژه را توضیح می‌دهد. README هر پروژه باید برای استفادهٔ مستقل و بدون این گزارش کافی باشد.

۱.۱ فهرست فعلی

پروژه	کاربرد	شواهد فعلی
DS18B20	خواندن دمای یک سنسور روی STM32F107VCT6 / PB10	ساخت و تست نرم‌افزاری؛ تست برد باقی است
قالب پایه	شروع مستقل برای پروژهٔ جدید	وضعیت محلی قالب بررسی شود

۲.۱ جای هر بخش در پروژه

```
ds18b20/
  DS18B20.ioc           # hardware configuration
  Core/                 # CubeMX-generated initialization
  Drivers/              # local ST HAL and CMSIS
  Application/          # board adapter and hardware demo
  Libraries/OneWire/    # portable wire protocol
  Libraries/DS18B20/    # portable temperature operation
  MDK-ARM/              # Keil project and startup
  platformio.ini        # GCC build and probe selection
  tests/                # simulated portable C tests
  tools/                # source synchronization/checks
  README.md             # standalone usage instructions
  PROJECT_STATUS.md     # evidence and unfinished work
docs/
  english/              # one complete report edition
  persian/              # matching report edition
  tools/                # report build and PDF verification
```

بخش Core میکروکنترلر را راه‌اندازی می‌کند. بخش Application رجیستر GPIO و کلاک را می‌شناسد. بخش

Libraries کد C قابل استفادهٔ دوباره برای صحبت با سنسور را نگه می‌دارد. با این جداسازی، کد دما هم در شبیه‌سازی و هم روی برد دیگر قابل استفاده است.

۳.۱ روش مطالعه و بازتولید

برای اولین نتیجه، مرحله‌های ۱ تا ۴ فصل سنسور را به ترتیب انجام دهید. مرحلهٔ ۵ دو تابع لازم برای پروژهٔ دیگر را آموزش می‌دهد و مراحل ۶ و ۷ نمونه و تغییر پایه را توضیح می‌دهند. مراحل ۸ تا ۱۰ پس از نتیجهٔ عملی به پیاده‌سازی می‌پردازند؛ می‌توانید بعداً به آن‌ها برگردید. ساخت موفق، تست شبیه‌سازی و اندازه‌گیری واقعی سه نتیجهٔ جدا هستند. مرحلهٔ نهایی، ثبت تست پذیرش برد است.

PDF دیتاشیت‌ها فقط مرجع محلی‌اند و وارد گیت نمی‌شوند. README و منابع گزارش، لینک رسمی و نسخهٔ منبع را حفظ می‌کنند. PDF نهایی گزارش و سورس قابل ویرایش در گیت نگهداری می‌شوند؛ فونت دانلودشده، تصویر بررسی و خروجی ساخت وارد گیت نمی‌شوند.

۲ خواندن اولین دما با سنسور

۱.۲ قرار است چه چیزی بسازیم؟

کار اول ساده است: یک سنسور را وصل کنیم و دمای آن را در کیل ببینیم. وقتی این مرحله درست شد، همان تابع خواندن را در پروژهٔ دیگر استفاده می‌کنیم. بعد سراغ داخل درایور می‌رویم. برای انجام مراحل اول لازم نیست اشاره‌گر تابع یا پروتکل باس را بلد باشید.

این نسخه برای **یک سنسور با تغذیهٔ مستقل در هر باس** است. دما را به سلسیوس می‌خواند و خطا را بررسی می‌کند. رزولوشن تغییر نمی‌کند؛ جست‌وجوی چند سنسور، نوشتن EEPROM و تغذیهٔ انگلی پیاده‌سازی نشده‌اند تا نسخهٔ اول قابل فهم بماند.

نتیجهٔ این فصل: مدار و نمونه را اجرا می‌کنید، نمایش ثابت Watch را از خطای سنسور تشخیص می‌دهید، از تابع خواندن استفاده می‌کنید، پایه را در CubeMX تغییر می‌دهید و می‌فهمید بایت‌های دما چگونه به یک عدد تبدیل می‌شوند.

مرجع سخت‌افزار، دیتاشیت رسمی سنسور با شمارهٔ 19-7487, Rev. 6, July 2019 است [۱].

۲.۲ مرحلهٔ ۱: سه سیم و یک مقاومت را بررسی کنیم

نمونهٔ موجود از STM32F107VCT6 استفاده می‌کند. سنسور را طبق جدول ببندید. پیش از انتخاب پایهٔ فیزیکی VDD و ،GND شکل پکیج را ببینید؛ نمای پایین قطعه را نباید نمای بالا در نظر گرفت.

سیگنال سنسور	اتصال به برد
VDD	تغذیهٔ ۳٫۳ ولت
GND	همان زمین میکروکنترلر
DQ	پایهٔ PB۱۰ برای داده
مقاومت pull-up	۴٫۷ کیلو اهم بین DQ و ۳٫۳ ولت

DQ تنها سیم داده است. GPIO باید drain open باشد؛ یعنی میکروکنترلر یا خط را پایین بکشد یا آن را رها کند. مقاومت خط رهاشده را بالا می‌برد. در این نمونه، pull-up داخلی GPIO جای مقاومت بیرونی نیست.

کلاک پروژه، کریستال خارجی ۸ مگاهرتز و کلاک پردازندهٔ ۷۲ مگاهرتز است. با کریستال واقعی برد تطبیق دهید. اتصال SWD از PA۱۳ و PA۱۴، زمین و ولتاژ مرجع پروگرامر تشکیل می‌شود. برای تست اول، نمایشگر یا ترمینال سریال لازم نداریم.

وب‌سایت · گیت‌هاب · لینکدین
یوتیوب · تلگرام · اینستاگرام

پیش از ادامه: یک سنسور داریم، VDD مستقل وصل است، زمین مشترک است، DQ به PB۱۰ می‌رود و مقاومت بیرونی وجود دارد.

۳.۲ مرحلهٔ ۲: بازکردن، ساخت و دانلود در کیل

۱. فایل ds18b20/MDK-ARM/DS18B20.uvprojx را باز کنید. اگر پروژه را جای دیگری کپی کرده‌اید، فایل همان کپی را باز کنید.

۲. پک STM۳۲F۱ نسخهٔ ۱.۴.۲ و Compiler ۶ نصب باشند. کامپایلر بررسی‌شده در این کار نسخهٔ ۲۴.۶ است.

۳. در Options for Target و بخش Debug پروگرامر واقعی را انتخاب کنید. برای J-Link همان J-Link و برای ST-Link درایور مناسب آن را انتخاب کنید. برای سنسور واقعی، Simulator انتخاب نشود.

۴. در Utilities بررسی کنید پروگرام با همان پروگرامر انجام شود. Build بگیرید و خطاهای ساخت را قبل از دیباگ برطرف کنید.

۵. firmware جدید را Download کنید و وارد Debug شوید. هر بار C عوض شد، دوباره Build و Download لازم است؛ ویرایش فایل، برنامهٔ روی برد را عوض نمی‌کند.

۶. هایbreakpoint مسیر خواندن را حذف یا غیرفعال و برنامه را آزاد اجرا کنید.

برنامه یک بار راه‌اندازی و بارها خواندن را انجام می‌دهد. تبدیل سنسور ممکن است حدود ۷۵۰ میلی‌ثانیه طول بکشد. صفر اولیه فقط مقدار شروع متغیر C است و اندازه‌گیری موفق نیست.

برای PlatformIO پوشهٔ پروژه را باز و محیط پروگرامر مناسب را اجرا کنید:

```
pio run -e stm32f107vc_jlink
pio run -e stm32f107vc_jlink -t upload
pio debug -e stm32f107vc_jlink
# For an ST-Link probe, use stm32f107vc_stlink.
```

Core PlatformIO باید نصب باشد. ساخت اول toolchain مشخص‌شده را دریافت می‌کند. firmware از HAL، startup CMSIS، و linker محلی همین پروژه استفاده می‌کند.

۴.۲ مرحلهٔ ۳: Watch برنامهٔ در حال اجرا را نشان بدهد

در جلسهٔ Watch Debug، ۱ را باز کنید و با دوبار کلیک روی ردیف خالی نام‌های زیر را اضافه کنید. از **نام متغیر سراسری** g_temperature_c استفاده کنید، نه متغیر محلی value داخل تابع.

عبارت	چه چیزی نشان می‌دهد؟
g_temperature_c	آخرین دمای موفق
g_temperature_status	نتیجهٔ آخرین تلاش کامل
g_temperature_valid	موفق‌بودن همان تلاش
g_measurement_busy	در حال انجام بودن خواندن جدید
g_successful_samples	تعداد خواندن‌های واقعاً موفق
g_sample_cycles	تعداد تلاش‌های کامل، حتی ناموفق
g_last_success_ms	تیک HAL در آخرین موفقیت

گزینهٔ View → Periodic Window Update را فعال کنید. کیل هنگام توقف اجرای برنامه، Watch را تازه می‌کند؛ به‌روزرسانی حین اجرا به این گزینه نیاز دارد [۲]. اگر خاموش باشد، breakpoint باعث دیده‌شدن مقدار جدید می‌شود چون توقف، نمایش را تازه می‌کند. این مشاهده ثابت نمی‌کند که breakpoint باعث اندازه‌گیری شده است.

هنگام بررسی این مشکل، همین گزینه در فایل تنظیمات محلی صفر بود. به یک تغییر داده شد و نام‌های Watch اضافه شدند. جلسهٔ باز کیل، تنظیماتی را که قبلاً بارگذاری کرده نگه می‌دارد؛ در همان جلسهٔ باز هم گزینهٔ منو را فعال کنید. فایل محلی uvoptx. وارد گیت نمی‌شود. برای تکرار تنظیمات، پروژه را ذخیره و کیل را ببندید، سپس از پوشهٔ پروژه اجرا کنید:

```
python tools/setup_keil_watch.py
```

این ابزار تنظیمات را پشتیبان می‌گیرد و انتخاب پروگرامر، هایWatch قبلی و هاbreakpoint را حفظ می‌کند. اگر جلسهٔ باز بعداً تنظیمات قدیمی را ذخیره کرد، پس از بستن کیل ابزار را دوباره اجرا کنید.

برای مثال، دمای ۲۴٫۵، وضعیت صفر، valid برابر true و شمارندهٔ موفق رو به افزایش، یک نتیجهٔ عادی موفق است. عدد دمای واقعی به مدار و محیط وابسته است. در اتاق ثابت ممکن است دما ثابت بماند؛ **شمارندهٔ موفق** نشان می‌دهد خواندن‌های جدید کامل می‌شوند یا نه.

اعتبار به آخرین تلاش *کامل‌شده* مربوط است و هنگام تبدیل بعدی دیده می‌ماند. busy همان تبدیل جدید را جدا نشان می‌دهد. اگر تلاش کامل‌شده خطا بدهد، valid برابر false و عدد قدیمی در حافظه باقی می‌ماند. در خطا از آن عدد به‌عنوان اندازه‌گیری تازه استفاده نکنید.

معیار عبور از این مرحله: در اجرای آزاد، شمارندهٔ موفق زیاد شود و وضعیت صفر بماند. اگر فقط صفحه ثابت است، پیش از تغییر درایور، refresh و نام متغیر سراسری را بررسی کنید.

۵.۲ مرحلهٔ ۴: نمایش خراب است یا خواندن سنسور؟

۱. مقدار فقط با breakpoint ظاهر می‌شود: اول تنظیم به‌روزرسانی دوره‌ای را بررسی کنید. صفحهٔ ثابت و CPU متوقف یک چیز نیستند.

۲. شمارندهٔ موفق زیاد می‌شود ولی دما تقریباً ثابت است: سنسور را کمی گرم کنید و چند نمونه را مقایسه کنید.

۳. تلاش‌ها زیاد می‌شوند ولی موفقیت‌ها نه: کد وضعیت را ببینید. حلقه اجرا می‌شود ولی تراکنش سنسور خطا دارد.

۴. دسترسی زنده هنوز ممکن نیست: برنامه را اجرا کنید و در فاصلهٔ ۲۵۰ میلی‌ثانیه‌ای بین تلاش‌ها Pause کنید. دو مشاهدهٔ شمارنده را مقایسه کنید. بعضی تنظیمات پروگرامر یا جلسهٔ دیباگ ممکن است دسترسی یکسان به حافظهٔ حین اجرا ندهند.

۵. شمارنده در توقف‌های امن هم پیش نمی‌رود: محل counter program را ببینید. ماندن در Handler Error یعنی مشکل راه‌اندازی؛ handler fault یعنی استثنای .CPU ماندن در Delay HAL به SysTick فعال و وقفهٔ باز نیاز دارد. فقط از عدد Watch نتیجه نگیرید درایور گیر کرده است.

برای دیدن دمای زنده، داخل تایم‌اسلات GPIO، تأخیر میکروثانیه یا polling توقف نگذارید. مکث دیباگر، ارتباط زمان‌دار را به‌هم می‌زند. breakpoint پس از خواندن کامل برای یک مشاهده مفید است، ولی روش معمول نمایش نیست.

وضعیت	معنی و اولین اقدام
صفر، موفق	دما قابل استفاده است.
۱، آرگومان نامعتبر	اشاره‌گر خروجی و اتصال هاcallback بررسی شود.
۲، نبود قطعه	VDD، زمین، DQ، مقاومت و پایهٔ واقعی بررسی شود.
۳، timeout	تغذیهٔ مستقل، کلاک و تأخیر بررسی شود.
۴، خطای CRC	اتصال، نویز، rise time، زمان‌بندی و توقف دیباگر بررسی شود.
۵، دادهٔ نامعتبر	نوع قطعه، بایت‌های ثابت و خط پایین بررسی شود.

۶.۲ مرحلهٔ ۵: دو تابع در پروژهٔ STM۳۲ خودمان

آداپتر سخت‌افزار، GPIO و تأخیر را پشت دو تابع نگه می‌دارد. تابع DS18B20_STM32_Init() یک بار آماده‌سازی می‌کند و تابع DS18B20_STM32_ReadTemperature(&temperature) هر بار یک تبدیل جدید

را شروع و نتیجه را می‌خواند. تابع **دو خروجی** دارد: مقدار برگشتی می‌گوید عملیات موفق بوده یا نه؛ خود دما با اشاره‌گر نوشته می‌شود. عبارت &temperature یعنی «آدرس این متغیر». نوشتن temperature = DS18B20_STM32_ReadTemperature(...) اشتباه است چون کد وضعیت، نه دما، در متغیر قرار می‌گیرد.

نوع float عدد اعشاری را نگه می‌دارد. نوع DS18B20_Status نتیجهٔ عملیات و ثابت DS18B20_OK موفقیت را نشان می‌دهد. بخش if فقط بعد از موفقیت از دما استفاده می‌کند.

```
#include "ds18b20_stm32.h"

/* After HAL, clock and GPIO initialization: */
DS18B20_STM32_Init();

/* From a foreground loop, with interrupts enabled: */
float temperature;
DS18B20_Status result = DS18B20_STM32_ReadTemperature(&temperature);
if (result == DS18B20_OK) {
    /* temperature can now be used. */
} else {
    /* Report the failure; do not use temperature here. */
}
```

این مثال کوتاه جای فراخوانی‌ها را نشان می‌دهد: include در بخش هدر، راه‌اندازی بعد از تنظیم برد و خواندن در حلقه. این قطعه، یک main کامل برای جایگزین‌کردن فایل تولیدشده نیست.

برای اضافه‌کردن به پروژهٔ STM۳۲F۱۰۷ دیگر:

۱. دو پوشهٔ قابل‌حمل Libraries/OneWire و Libraries/DS18B20 را کپی کنید.

۲. فایل‌های Application/Inc/ds18b20_stm32.h و Application/Src/ds18b20_stm32.c را کپی کنید.

۳. دو فایل C کتابخانه و فایل C آداپتر را به ساخت اضافه کنید. سه پوشهٔ Inc مربوط به آن‌ها در مسیر هدر کامپایلر باشد.

۴. برچسب GPIO خروجی drain open را در CubeMX برابر ONE_WIRE_1 قرار دهید. آداپتر ماکروهای تولیدشدهٔ پایه را می‌خواند. کلاک درست باشد و هنگام خواندن وقفه فعال بماند.

۵. پس از تنظیم برد، یک بار Init و در حلقهٔ معمول برنامه ReadTemperature را با بررسی وضعیت اجرا کنید.

این آداپتر برای همین Cortex-M۳ و رجیسترهاست. MCU دیگر آداپتر، startup vendor، و linker مناسب خودش را می‌خواهد؛ دو کتابخانهٔ قابل‌حمل به هدر STM۳۲ نیاز ندارند.

۷.۲ مرحلهٔ ۶: نمونهٔ کامل اجرایی را بفهمیم

کد زیر همان برنامه‌ای است که firmware نمونه اجرا می‌کند:

```
#include "app.h"
#include "ds18b20_stm32.h"
#include "main.h"

volatile float g_temperature_c;
volatile bool g_temperature_valid;
volatile bool g_measurement_busy;
volatile DS18B20_Status g_temperature_status = DS18B20_NO_DEVICE;
volatile uint32_t g_sample_cycles;
volatile uint32_t g_successful_samples;
volatile uint32_t g_last_success_ms;

void App_Init(void)
{
    DS18B20_STM32_Init();
}

void App_Process(void)
{
    float value;
    g_measurement_busy = true;
    DS18B20_Status status = DS18B20_STM32_ReadTemperature(&value);
    /* Publish the latest completed attempt. Keep its validity visible while
     * the next conversion is running; busy reports that conversion separately.
    */
    g_temperature_status = status;
    g_temperature_valid = (status == DS18B20_OK);
    if (g_temperature_valid) {
        g_temperature_c = value;
        ++g_successful_samples;
        g_last_success_ms = HAL_GetTick();
    }
    ++g_sample_cycles;
    g_measurement_busy = false;
    HAL_Delay(250U);
}
```

از این ترتیب بخوانید: تابع راه‌اندازی، Init آداپتر را صدا می‌زند. پردازش، busy را فعال می‌کند، دما می‌خواهد، وضعیت را ثبت و فقط نتیجهٔ موفق را منتشر می‌کند. سپس شمارندهٔ تلاش کامل را زیاد و busy را پاک می‌کند و پیش از تلاش بعدی مکث دارد.

متغیر موقت value فقط برای همان فراخوانی است. متغیرهای g_... سراسری‌اند و Watch آدرس ثابت آن‌ها

در RAM را پیدا می‌کند. volatile تضمین می‌کند کامپایلر نوشتن را انجام دهد؛ گزینهٔ refresh نرم‌افزار را فعال نمی‌کند. این چند فیلد جدا نیز یک پیام همگام RTOS نیستند.

در موفقیت، شمارندهٔ موفق و تیک آخرین موفقیت عوض می‌شوند. در خطا، فقط شمارندهٔ تلاش و وضعیت و اعتبار عوض می‌شوند؛ دمای قبلی و زمان آخرین موفقیت دست‌نخورده می‌مانند. تست جدید برنامه همین توالی و برگشت از خطا را بررسی می‌کند.

main تولیدشده فقط سه اتصال به برنامه دارد:

```
/* USER CODE BEGIN Includes */
#include "app.h"
/* USER CODE END Includes */

/* USER CODE BEGIN 2 */
App_Init();
/* USER CODE END 2 */

/* USER CODE BEGIN 3 */
App_Process();
/* USER CODE END 3 */
```

اولی در بخش include، دومی پس از تنظیم HAL و کلاک و GPIO و سومی داخل حلقهٔ بی‌نهایت تولیدشده است. آکولادهای حلقه حفظ شوند. یا نمونهٔ پردازش موجود را استفاده کنید یا حلقهٔ خواندن خودتان را؛ دو خواننده هم‌زمان روی یک باس اجرا نکنید.

۸.۲ مرحلهٔ ۷: تغییر پایه یا شروع از کپی پروژه

برای نمونهٔ مستقل کامل، کل پوشهٔ پروژهٔ سنسور را کپی کنید. برای درایور جدید، قالب خام را کپی کنید؛ از آن به‌عنوان وابستگی ساخت استفاده نکنید. هر پروژه vendor IOC، و تنظیمات ساخت محلی دارد.

برای تغییر GPIO همین نمونه:

۱. IOC را باز کنید. پایه را جابه‌جا کنید ولی برچسب ONE_WIRE_1 و خروجی open، drain مقدار اولیهٔ high، بدون pull و سرعت high را حفظ کنید.

۲. کلاک و SWD درست بمانند. در Manager Project حفظ user code، کپی vendor محلی، MDK-ARM V۵ و نام پروژهٔ DS۱۸B۲۰ تنظیم شوند. Generate در همان پوشه انجام شود [۳].

۳. سه اتصال برنامه در CODE USER را کنترل کنید. فایل‌های Application و Libraries از Core تولیدشده جدا هستند.

۴. دستورهای زیر، سپس Build کیل و firmware Download جدید را انجام دهید.

```
python tools/sync_keil.py
python tools/check_project.py
python tests/run_tests.py
pio run -e stm32f107vc_jlink -e stm32f107vc_stlink
```

hook بعد از تولید تلاش می‌کند sync و check را خودکار انجام دهد. اگر Python در محیط CubeMX نیست، دستی اجرا کنید. sync، فایل‌های برنامه، آداپتر و کتابخانه را به کیل اضافه می‌کند؛ فایل پروژهٔ تولیدشده به‌تنهایی تمام فایل‌های C دست‌نویس را نمی‌شناسد.

تست انتقال پایهٔ انجام‌شده: در کپی موقت، PB۱۰ به PB۱۱ منتقل و پروژه Generate و با هر دو toolchain ساخته شد؛ literal پایه در آداپتر عوض نشد. نمونهٔ تحویلی همچنان PB۱۰ است.

۹.۲ مرحلهٔ ۸: توالی سنسور را بدون خواندن کل کد دنبال کنیم

دو درخواست از سنسور را تصور کنید: «الان اندازه بگیر» و بعد «نتیجه چه بود؟». هر دو با reset شروع می‌شوند تا توجه سنسور جلب شود.

۱. reset و presence بررسی شود. بدون پاسخ، وضعیت نبود قطعه برگردد.

۲. فرمان $0xCC$ یعنی انتخاب تنها سنسور و فرمان $0x44$ یعنی شروع تبدیل دما ارسال شود.

۳. تا پایان تبدیل polling شود و بین هاbusy انتظار باشد. با تمام‌شدن بودجه، انتظار خاتمه پیدا کند.

۴. دوباره reset و فرمان‌های $0xCC$ و $0xBE$ برای خواندن scratchpad، یعنی ناحیهٔ ۹ بایتی نتیجه، ارسال شوند.

۵. بایت بررسی کنترل و دما decode شود. در موفقیت OK و در خطا، دمای خروجی کاربر بدون تغییر بماند.

فرمان انتخاب عمومی دلیل شرط تک‌سنسور است. چند سنسور هم‌زمان پاسخ می‌دهند و این نسخه تعداد یا آدرس فردی آن‌ها را پیدا نمی‌کند. VDD مستقل، polling حین تبدیل را ممکن می‌کند؛ تغذیهٔ انگلی مدار و روش دیگری می‌خواهد.

دمای خام در دو بایت اول، کم‌ارزش‌تر در ابتدا، قرار دارد. برای بایت‌های $01\ 91$ مقدار $0x0191$ یا ۴۰۱ به دست می‌آید. واحد خام یک‌شانزدهم درجه است؛ ۴۰۱ تقسیم بر ۱۶ برابر ۲۵٫۰۶۲۵ درجه می‌شود. نمونهٔ منفی $FF5E$ به منفی ۱۶۲ تقسیم بر ۱۶، یعنی منفی ۱۰٫۱۲۵ درجه تبدیل می‌شود.

CRC خرابی بایت‌ها را بررسی می‌کند. بایت‌های ثابت هم کنترل می‌شوند چون ۹ صفر، CRC صفر معتبر دارد؛ فقط CRC خط پایین را نمی‌گیرد. رزولوشن تعداد بیت‌های معنادار پایین را مشخص می‌کند و بیت تعریف‌نشده

قبل از تبدیل حذف می‌شود. ۸۵ درجهٔ معتبر پذیرفته می‌شود چون ممکن است دمای واقعی باشد؛ تکرار نامعقول آن، بررسی تغذیه و زمان تبدیل می‌خواهد.

۱۰.۲ مرحلهٔ ۹: تابع قابل‌حمل را بخش‌به‌بخش بخوانیم

این کل عملیات خواندن دماست؛ فایل C آن هدر MCU ندارد:

```c
#include "ds18b20.h"
DS18B20_Status DS18B20_ReadTemperature(const DS18B20 *sensor, float *celsius)
{
    uint8_t scratchpad[9];
    uint16_t bits;
    int32_t raw;
    uint32_t elapsed = 0;
    const OneWire_Bus *bus;
    if (!sensor || !celsius || !sensor->delay_ms ||
        !OneWire_IsValid(&sensor->bus)) return DS18B20_INVALID_ARGUMENT;
    bus = &sensor->bus;
    if (!OneWire_Reset(bus)) return DS18B20_NO_DEVICE;
    OneWire_WriteByte(bus, 0xCC); /* Skip ROM: one sensor only. */
    OneWire_WriteByte(bus, 0x44); /* Convert T. */
    while (!OneWire_ReadBit(bus)) {
        if (elapsed >= 800U) return DS18B20_TIMEOUT;
        sensor->delay_ms(bus->context, 1U);
        ++elapsed;
    }
    if (!OneWire_Reset(bus)) return DS18B20_NO_DEVICE;
    OneWire_WriteByte(bus, 0xCC);
    OneWire_WriteByte(bus, 0xBE); /* Read scratchpad. */
    for (uint8_t i = 0; i < 9U; ++i) scratchpad[i] = OneWire_ReadByte(bus);
    if (OneWire_Crc8(scratchpad, 8U) != scratchpad[8]) return DS18B20_CRC_ERROR;
    /* Nine zero bytes pass CRC: verify fixed bytes and configuration too. */
    if ((scratchpad[4] & 0x9FU) != 0x1FU || scratchpad[5] != 0xFFU ||
        scratchpad[6] != 0x0CU || scratchpad[7] != 0x10U)
        return DS18B20_INVALID_DATA;
    bits = (uint16_t)scratchpad[0] | ((uint16_t)scratchpad[1] << 8U);
    /* Ignore undefined low bits at 9/10/11-bit resolution. */
    bits &= (uint16_t)(0xFFFFU << (3U - ((scratchpad[4] >> 5U) & 3U)));
    raw = (bits & 0x8000U) ? (int32_t)bits - 65536 : (int32_t)bits;
    if (raw < -55 * 16 || raw > 125 * 16) return DS18B20_INVALID_DATA;
    *celsius = (float)raw / 0.16f;
    return DS18B20_OK;
}
```

بلوک اول اشاره‌گرها و callback را کنترل می‌کند. بعد تبدیل شروع می‌شود. حلقهٔ while انتظار محدود دارد.

reset دوم خواندن نتیجه را شروع می‌کند. حلقهٔ for نه بایت را می‌گیرد. CRC و مقدار ثابت، دادهٔ خراب را رد می‌کنند. در پایان دو بایت ترکیب، علامت منفی و محدوده بررسی و سلسیوس در متغیر کاربر نوشته می‌شود.

۸۰۰ انتظار میلی‌ثانیهٔ اسمی deadline ساعت واقعی نیست. تایم‌اسلات، وقفه و گردشدن تیک HAL سربار دارند؛ timeout نمونه در busy دائمی می‌تواند حدود ۱٫۷ ثانیه شود. فراخوانی blocking است: اجرای اصلی انتظار می‌کشد ولی وقفه بین بخش‌های حساس می‌تواند اجرا شود. در ISR یا با وقفهٔ بسته صدا نزنید.

۱۱.۲ مرحلهٔ ۱۰: وقتی خواستیم منتقل کنیم، آداپتر را بفهمیم

callback یک تابع عادی است که آدرسش به تابع دیگر داده می‌شود. به‌جای واردکردن رجیستر STM۳۲ به کتابخانه، توابعی با معنی «پایین بکش»، «آزاد کن»، «بخوان» و «صبر کن» به آن می‌دهیم. آداپتر آن‌ها را فراهم می‌کند.

```
/* Inside the STM32 adapter: */
static void low(void *context)
{
    (void)context;
    ONE_WIRE_1_GPIO_Port->BRR = ONE_WIRE_1_Pin;
}
```

static این تابع کمکی را خصوصی همان فایل C نگه می‌دارد. context در برد دیگر می‌تواند شیء پورت و پایه را منتقل کند؛ نمونهٔ پایهٔ ثابت به آن نیاز ندارد. ماکرو GPIO از CubeMX است، بنابراین تغییر پایه به ویرایش این تابع نیاز ندارد.

هفت تابع، هفت کار جدا دارند: پایین‌کشیدن، آزادکردن، خواندن سطح، تأخیر میکروثانیه، ذخیره و بستن وقفه، بازیابی وقفه و تأخیر میلی‌ثانیه. تنظیم باس آدرس آن‌ها را نگه می‌دارد:

```
void DS18B20_STM32_Init(void)
{
    CoreDebug->DEMCR |= CoreDebug_DEMCR_TRCENA_Msk;
    DWT->CYCCNT = 0;
    DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
    const DS18B20 binding = {
        {NULL, low, release, read_line, delay_us, lock, unlock}, delay_ms
    };
    sensor = binding;
    release(NULL);
}

DS18B20_Status DS18B20_STM32_ReadTemperature(float *celsius)
{
    return DS18B20_ReadTemperature(&sensor, celsius);
}
```

شمارندهٔ سیکل DWT، تأخیر کوتاه را می‌سازد؛ در ۷۲ مگاهرتز هر میکروثانیه ۷۲ سیکل است. HAL انتظار میلی‌ثانیه را می‌دهد. توابع وقفه وضعیت ذخیره‌شده را برمی‌گردانند و وقفه را بی‌قید فعال نمی‌کنند.

لایهٔ سیم بایت را از کم‌ارزش‌ترین بیت می‌فرستد. reset، ۴۸۰ میکروثانیه پایین می‌کشد و ۷۰ پس از آزادکردن presence را می‌خواند. ارسال یک، ۶ پایین و ۶۴ آزاد؛ صفر، ۶۰ پایین و ۱۰ آزاد است. خواندن، ۳ پایین، ۱۰ پس از آزادکردن تا نمونه و ۵۳ recovery درخواست می‌کند. سربار GPIO و فراخوانی روی برد اندازه‌گیری شود. reset حدود ۵۵۰ میکروثانیه به‌اضافهٔ سربار، وقفه را می‌بندد؛ برای deadline سخت‌تر طراحی زمان دیگری لازم است.

برای MCU دیگر، آداپتر را بنویسید و زمان‌ها را اندازه بگیرید؛ دو فایل C قابل‌حمل حفظ شوند. برای چند سنسور یا تبدیل غیرمسدودکننده ابتدا رفتار API و تست تعریف شود. این قابلیت‌ها عمداً خارج از درس اول هستند.

۱۲.۲ مرحلهٔ ۱۱: برد واقعاً چه چیزی را ثابت کرد؟

صاحب پروژه گزارش کرده firmware قبلی را با کیل دانلود و اجرا کرده و هنگام breakpoint دما دیده شده است. این بازخورد اولیهٔ سخت‌افزار مفید است، اما هنوز refresh پیوسته، دقت دما، قطع و وصل و زمان موج واقعی را ثابت نمی‌کند. تنظیم اصلاح‌شده باید در جلسهٔ واقعی دوباره امتحان شود.

این موارد جدا ثبت شوند:

۱. refresh فعال: موفقیت‌ها بدون breakpoint تراکنش زیاد شوند.

۲. گرم و سردکردن: چند دمای موفق متناسب تغییر کنند.

۳. قطع و وصل: خطا اعتبار را پاک و اتصال مجدد خواندن را برگرداند.

۴. مقایسه با دماسنج مرجع و بررسی DQ زیر بار وقفهٔ واقعی انجام شود.

۵. نسخه، ولتاژ، کریستال، مقاومت، سیم، پروگرامر، دما و موج ثبت شوند.

مشاهدات واقعی در فایل وضعیت ثبت شوند: PROJECT_STATUS.md. ساخت و شبیه‌سازی شواهد نرم‌افزارند؛ پذیرش برد به مشاهدات بالا نیاز دارد. سه تست C از Unicorn Python و GCC ARM مربوط به PlatformIO استفاده می‌کنند.

۳ افزودن درایور مستقل بعدی

۱.۳ شروع از کپی قالب پایه

قالب خام را کامل در پوشه‌ای به نام قطعه کپی کنید. target IOC، کیل و خروجی را نام‌گذاری و سخت‌افزار را در CubeMX تنظیم کنید. پروژهٔ کپی‌شده باید فایل‌های vendor و ساخت خود را داشته باشد. پس از کپی، مسیر هدر یا سورس به قالب یا پروژهٔ مجاور اشاره نکند.

ابتدا دیتاشیت رسمی را بخوانید و شماره، نسخه و URL رسمی را در README ثبت کنید. PDF دانلودشده محلی بماند. کوچک‌ترین عملیات کاربردی، واحد خروجی، رفتار timeout و سیاست خطا را پیش از API بزرگ‌تر تعریف کنید.

۲.۳ ترتیب توسعهٔ قابل تکرار

۱. پایه، کلاک و فرض تغذیه را در IOC محلی تنظیم کنید.

۲. هدر و پیاده‌سازی کوچک قابل‌حمل بسازید. HAL و رجیستر فقط در آداپتر باشند.

۳. نمونهٔ کامل اجرایی برد با مقدار و خطای قابل مشاهده اضافه کنید.

۴. اتصال برنامه در CODE USER باشد؛ در کپی موقت Generate و حفظ کد بررسی شود.

۵. کیل sync شود؛ ساختار، ساخت کیل و PlatformIO و تست معنادار پروتکل و خطا بررسی شوند.

۶. README مستقل، ابتدا انگلیسی و سپس فارسی نوشته و وضعیت با شواهد واقعی به‌روز شود.

۷. یک فصل به هر نسخهٔ مرکزی گزارش و یک سطر به فهرست مخزن افزوده و هر دو PDF ساخته شود.

۸. پذیرش واقعی برد انجام و اندازه‌گیری‌ها بدون حذف شواهد نرم‌افزار اضافه شوند.

۳.۳ افزودن فصل به همین گزارش

در دو پوشهٔ chapters فایل شماره‌دار متناظر بسازید و دستور input آن‌ها را پیش از فصل توسعه به هر دو main اضافه کنید. بخش تماس آخر بماند. فصل قطعه همان ترتیب فصل دما را دارد: هدف، اتصال، اولین اجرا، پروتکل، API، دلیل پیاده‌سازی، آداپتر، Generate، خطا، پذیرش و استفادهٔ دوباره. منبع رسمی به هر دو فایل bib اضافه و در محل استفاده cite شود.

موتور هر دو نسخه، XeLaTeX زبان فارسی XePersian و منابع IEEEtran و BibTeX هستند. تم مشترک، فاصلهٔ متن ۱٫۵، آیکون کوچک لینک و شش مقصد نویسنده در config تنظیم می‌شوند. اصطلاح لاتین فارسی با lr یا محیط LTR و در خط مستقل سورس قرار گیرد تا ویرایش آسان بماند.

از ریشهٔ مخزن اجرا کنید:

```
python docs/tools/build_reports.py
python docs/tools/verify_reports.py
```

ابزار ساخت یک بار، XeLaTeX سپس BibTeX و دو بار دیگر XeLaTeX را اجرا می‌کند. verifier، نام نویسنده، شش مقصد در پاورقی هر صفحه و محدودهٔ صفحه را بررسی و صفحات را برای مشاهده render می‌کند. contact sheet و صفحه‌های کامل را هم ببینید؛ تست خودکار لینک به‌تنهایی کیفیت صفحه‌آرایی را ثابت نمی‌کند. فونت و تصویر بررسی ignored هستند؛ PDF نهایی tracked است.

منابع

[1] Analog Devices / Maxim Integrated, *DS18B20 Programmable Resolution 1-Wire Digital Thermometer*, Jul. 2019, rev. 6. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf

[2] Arm / Keil, *uVision User's Guide: Watch Window*, accessed October 12, 2026. [Online]. Available: https://www.keil.com/support/man/docs/uv4cl/uv4cl_db_dbg_watchwin.htm

[3] STMicroelectronics, *STM32CubeMX User Manual and Command-Line Interface*, uM1718, accessed October 12, 2026. [Online]. Available: https://dev.st.com/stm32cube-docs/stm32cubemx/6.18.0/en/docs/markup/CubeMX_CLI.html

دربارهٔ نویسنده و راه‌های ارتباطی

علیرضا خواجه‌وندی

این مخزن و راهنمای آموزشی آن با نام من نگهداری می‌شوند. مشخصات قطعات و کد vendor به منابع اصلی خود ارجاع داده می‌شوند.

وب‌سایت https://alirezaaies.github.io/

گیت‌هاب https://github.com/alirezaaies

لینکدین https://www.linkedin.com/in/alirezakhajehvandi

یوتیوب https://www.youtube.com/@AlirezaAIES

تلگرام https://t.me/AlirezaAIES

اینستاگرام https://www.instagram.com/alireza.aies